

Week 6 — Transaction malleability, SegWit, multisig, and Merkle trees

1) Recap: legacy Bitcoin transaction outputs

The lecture starts by revisiting the legacy Bitcoin transaction and reminding us of a key rule:

If ScriptPubKey(ScriptSig), then create output. The slide shows the output being locked to Babar's public key hash and another output locked back to Alia's public key hash, meaning payment plus change.

This recap matters because all of Week 6 builds on the idea that:

- old outputs are locked by a script,
- the spender provides unlocking data,
- if validation succeeds,
- new outputs are created.

So one very important mental model to keep all week is:

Bitcoin spends old outputs and creates new outputs.

That remains true whether the transaction is:

- legacy,
 - SegWit,
 - or multisig.
-

2) Multiple inputs and outputs

The lecture then shows that a Bitcoin transaction can have **multiple inputs and multiple outputs**. The page 4 diagram visually shows that a transaction may combine value from multiple earlier outputs and may create separate outputs for different recipients or for change.

2.1 Why would one transaction need multiple inputs?

Because one old UTXO may not be enough to cover the new payment.

Suppose Alia wants to pay 0.5 BTC, but she only has:

- one UTXO of 0.2 BTC
- another UTXO of 0.3 BTC

Then the transaction can take both as inputs.

So multiple inputs let the user combine spendable pieces of value.

2.2 Why multiple outputs?

Because one payment may produce several results:

- one output to the payee,
- one output back to the sender as change,
- possibly more outputs to other recipients.

So Bitcoin transactions are not limited to “one sender, one receiver, one output.”

2.3 Easy analogy

Think of paying with cash using:

- two smaller notes as input,
- one payment to the shop,
- and some change back to yourself.

That is basically what multiple inputs and outputs are doing in digital form.

3) Unspendable output and OP_RETURN

The lecture then shows an **unspendable output** with:

- value = 0 BTC
- scriptPubKey = **OP_RETURN** <anything>

This is a very interesting concept.

3.1 What is an unspendable output?

It is an output intentionally created so that nobody can later spend it.

Normally Bitcoin outputs are made to be spendable by some rightful owner. But with **OP_RETURN**, the output is made deliberately unspendable.

3.2 Why would anyone do that?

Usually to store or attach small pieces of data on-chain rather than to transfer spendable value.

So `OP_RETURN` outputs are more like “record this message/data pointer” than “give spendable money.”

3.3 Why value = 0 BTC?

Because if the output is unspendable, you would not want to trap valuable BTC in it.

So the lecture’s example uses zero value to show this output is for information, not for future spending.

3.4 Big takeaway

Not every Bitcoin output is meant to become future money.

Some outputs are intentionally non-spendable and are used for special purposes.

4) Legacy transaction example revisited

The lecture then returns to a concrete legacy transaction example with:

- previous transaction ID
- index
- ScriptSig containing signature and public key
- two outputs
- output values
- P2PKH-style ScriptPubKeys
- transaction ID.

This is the same structural idea as Week 5, but now sir is using it as the base for explaining more advanced issues.

4.1 Why keep returning to the same example?

Because it helps you compare:

- normal legacy behavior
- how signatures are encoded
- what malleability changes
- how SegWit restructures it

So think of this example as the lecture's "working specimen."

5) scriptSig and ECDSA signatures

Now the lecture zooms into the **signature** inside `scriptSig`. It says:

- the signature is created using **ECDSA** (Elliptic Curve Digital Signature Algorithm)
- an ECDSA signature is a pair (`r`, `s`)
- both `r` and `s` are integers
- and the encoded signature format is:
`30 <total length> 02 <r length> <r bytes> 02 <s length> <s bytes> <sighash byte>`

This looks scary at first, but it is manageable.

5.1 What is ECDSA?

ECDSA is the digital signature algorithm Bitcoin used for legacy signatures.

You do not need to derive the elliptic curve math for this course from these slides. The important thing is:

Bitcoin signature = structured cryptographic proof created with the private key.

5.2 What are `r` and `s`?

An ECDSA signature is represented as two numbers:

- `r`
- `s`

Those two numbers together form the signature content.

5.3 Why is there a byte format around them?

Because the signature has to be serialized into bytes so it can be put into the transaction.

That is why the lecture shows the DER-like structure:

- `30` for sequence
- integer tag `02`
- lengths

- actual bytes
- and then the sighash byte.

5.4 What is the sighash byte?

The lecture says it indicates which parts of the transaction were hashed and signed. It gives examples:

- `0x01` = all inputs and all outputs hashed and signed
- `0x02` = all inputs hashed and signed but no output.

That means the signature can have rules about what exactly it commits to.

For your level, the main thing to remember is:

the signature includes not only `r` and `s`, but also a flag that says what part of the transaction the signature covers.

6) Why do leading zeros matter in signature encoding?

The lecture shows two signature encodings:

- one with total length `0x44`
- another with total length `0x45`
where in one case `r` is 32 bytes, and in another case it is 33 bytes because of a leading zero byte.

This is the key setup for transaction malleability.

6.1 What is happening here?

Even though the mathematical signature is essentially the same, its byte representation can sometimes be encoded in more than one valid way.

For example, adding a leading zero byte to an integer can still represent the same value correctly in the signature structure.

6.2 Why is this a problem?

Because if transaction ID depends on the serialized bytes, then changing the byte format changes the txid — even if the actual spending meaning does not change.

That is exactly the malleability issue.

7) Transaction malleability — the core idea

This is one of the biggest Week 6 topics.

The lecture says:

- a transaction's ID (txid) can be changed without altering its actual effect on the blockchain
- txid is computed as the hash of the serialized transaction
- in legacy Bitcoin, this serialization included signatures inside `scriptSig`
- ECDSA signatures allow multiple valid DER encodings
- so someone can slightly modify signature byte format, producing a different txid
- even though the transaction still spends the same inputs and creates the same outputs.

This is extremely important.

7.1 What is txid?

Transaction ID is the hash-based identifier of the transaction.

Wallets and users use it to track transactions.

7.2 Why can it change?

Because in legacy Bitcoin, the txid includes the serialized `scriptSig`, and `scriptSig` contains the signature bytes.

If you change the byte encoding of the signature in a still-valid way, the serialized transaction changes.

If serialized transaction changes, its hash changes.

So:

same logical transaction effect

but

different txid

7.3 What does “actual effect” mean?

It means the transaction still:

- spends the same inputs,
- pays the same outputs,

- transfers the same value.

So the economic effect is the same.

Only the ID changes.

7.4 Why is that bad?

Because users and software often assume txid is a stable identifier of the transaction they sent.

Malleability breaks that assumption.

8) Why transaction malleability is a problem

The lecture lists several consequences:

- unreliable transaction IDs
- wallets may not recognize that confirmed tx is the same one they sent
- double-spend risks
- attackers can alter a pending transaction's signature and rebroadcast it
- network confusion
- multiple versions of "same" transaction may circulate with different IDs
- breaks Layer-2 protocols
- off-chain protocols need exact txids
- malleability prevents pre-building transactions that depend on earlier txids.

Let's unpack each one.

8.1 Unreliable txids

If txid can change due to harmless signature-encoding differences, then it is no longer a perfectly stable name for the transaction.

That is bad for wallets and tracking software.

8.2 Wallet confusion

Suppose your wallet broadcasts a transaction and expects a specific txid.

If miners confirm a modified-but-equivalent version with a different txid, the wallet may think:

- my original tx disappeared
- or something weird happened

That causes user confusion.

8.3 Rebroadcast attack flavor

An attacker can take your pending transaction, alter the signature encoding without changing meaning, and rebroadcast it.

If the altered version gets mined first, then the txid users were expecting is not the one confirmed.

8.4 Layer-2 breakage

This is huge.

Off-chain systems often want to create transactions that refer to the txid of some earlier transaction.

But if that earlier txid can still change, then dependent constructions break.

So transaction malleability was a serious obstacle for early payment channel and Lightning-style designs.

This is why fixing it was very important.

9) Segregated Witness (SegWit) — the solution idea

The lecture explains the fix:

- `scriptSig = <ECDSA signature, public key>` acts as witness proving authorization and ownership
- to prevent malleability, we need to **segregate this witness from the transaction input**
- this leads to **Segregated Witness (SegWit)**
- txid is not created using `scriptSig`.

This is one of the most important upgrade ideas in Bitcoin history.

9.1 What is “witness”?

Witness means the data that proves the spender had the right to spend.

In legacy, this proof was placed inside `scriptSig` in the main serialized transaction body.

9.2 What does “segregate” mean?

Separate it.

Move the witness data out of the place where it affects txid calculation.

9.3 Main benefit

If txid no longer includes the mutable signature bytes, then harmless changes to witness encoding will not change txid.

So the transaction ID becomes much more stable.

9.4 The big idea in one line

SegWit fixes transaction malleability by moving signature/witness data outside the part used to compute txid.

That is the key exam sentence.

10) Legacy vs SegWit transaction structure

The lecture includes a side-by-side diagram comparing:

Legacy P2PKH

- version
- input count
- inputs including prev txid, vout index, ScriptSig length, ScriptSig, sequence
- output count
- outputs
- locktime

SegWit P2WPKH

- version
- marker 0x00
- flag 0x01
- input count
- inputs with empty ScriptSig
- outputs
- witnesses per input
- locktime

This page is very important visually.

10.1 What changes in SegWit?

The key structural change is:

- ScriptSig becomes empty in the input
- witness data is stored in a separate witness section

10.2 Why marker and flag?

They indicate the transaction uses SegWit format.

10.3 Why is this useful?

Because the main body used for txid is cleaner and excludes witness mutability issues.

So structure changed to improve correctness and support future scalability.

11) SegWit P2WPKH transaction format

The lecture then shows a concrete SegWit transaction with:

- version
- marker 00
- flag 01
- one input
- previous txid
- vout index
- empty ScriptSig
- one output
- ScriptPubKey of form 00 14 <20-byte pubKeyHash>
- witness items:
 - count
 - signature
 - public key.

11.1 What does P2WPKH mean?

Pay to Witness Public Key Hash

This is basically a SegWit version of the “pay to pubkey hash” idea.

11.2 What is special about the scriptPubKey?

Instead of old-style opcodes directly written there, it uses a compact witness program:

- SegWit version 0
- length
- 20-byte pubkey hash.

11.3 Where are signature and public key now?

Not in ScriptSig.

They are in the **witness** section.

That is the whole structural point of SegWit.

12) “No op codes!” — but then how does validation work?

The lecture says for SegWit P2WPKH, ScriptSig is empty and there are “No OP Codes!” in the visible scriptPubKey form. Then it says Bitcoin internally expands the previous output’s scriptPubKey into a **virtual script**:

```
OP_DUP OP_HASH160 <pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG
```

This is extremely important.

12.1 What is going on?

Even though SegWit outputs look simpler on-chain, Bitcoin still knows how to interpret them.

Internally, for validation, it reconstructs the equivalent logical checks.

So the actual authorization logic is still similar to P2PKH:

- provided public key must hash correctly
- signature must verify

12.2 Why do this?

Because the system wants the benefits of SegWit structure while preserving meaningful spend-validation logic.

So SegWit changes where the witness data lives, not the core idea that ownership and signature must be checked.

13) P2WPKH execution on the stack

The lecture then spends many slides walking through stack execution for SegWit P2WPKH.

The main story is:

- ScriptSig is empty
- witness provides signature and public key
- previous output's SegWit scriptPubKey is expanded into a virtual P2PKH-like script
- witness items are pushed to the stack
- opcodes run
- the final result becomes **True**.

This is actually simpler than it looks.

13.1 Step 1: Empty ScriptSig

Unlike legacy, the unlocking data is not in ScriptSig.

So nothing useful is pushed from ScriptSig.

13.2 Step 2: Expand virtual script

Bitcoin interprets the witness-program output and reconstructs the equivalent checking script.

13.3 Step 3: Push witness items

The witness section supplies:

- signature
- public key

These are placed on the stack for validation.

13.4 Step 4: Execute checks

Then the same kind of logic runs:

- duplicate pubkey
- hash it

- compare with expected hash
- verify signature

13.5 Step 5: Final True

If all checks pass, the stack ends in **True**, which means the spend is valid. The final slide in this execution sequence explicitly shows **True** on the stack.

13.6 Main lesson

SegWit changed **where** witness lives, but the ultimate ownership test still asks:

- is the provided pubkey the right one?
 - is the signature valid?
-

14) Resulting outputs in SegWit P2WPKH

The lecture then shows the SegWit transaction creating outputs locked to:

- Babar
- Alia (change)

So the same old spending logic remains:

- valid input authorization
- then creation of new spendable outputs

Again, the architecture of Bitcoin remains UTXO-based:
consume old outputs, create new ones.

Lecture 12 — Multisig

Now the lecture moves to another major topic: **multisig transactions**.

15) What is a multisig transaction?

The lecture says:

- multisig means multiple signatures
- a Bitcoin transaction requires M-of-N keys to authorize spending
- example: 2-of-3 means any 2 signatures out of 3 keys are enough

It also gives use cases:

- security
- shared accounts
- escrow
- Lightning Network 2-of-2 outputs.

This is a very important topic.

15.1 What does M-of-N mean?

It means:

- there are N total authorized public keys
- but only M signatures are needed to spend

Example:

2-of-3 means three possible keys exist, but any two valid signatures are enough.

15.2 Why is this useful?

Because control can be shared.

You do not want one single key to be the only point of failure.

15.3 Examples

Security

If one key is lost or stolen, funds may still be protected.

Shared accounts

A company treasury may require multiple approvals.

Escrow

Buyer, seller, and arbitrator can form a 2-of-3 setup.

Lightning

Payment channels often use 2-of-2 multisig.

So multisig is about shared or conditional control.

16) How multisig works

The lecture explains the three phases:

Locking (output side)

Bitcoin is sent to a script like:

```
OP_2 <PubKey1> <PubKey2> <PubKey3> OP_3 OP_CHECKMULTISIG
```

This script is hashed into an address, often P2SH or P2WSH.

Unlocking (input side)

The spender provides:

- M valid signatures
- the witness/redeem script proving the locking conditions

Verification

Bitcoin checks whether the provided signatures match the required keys.

16.1 Locking script meaning

The output says:

“To spend me, you must satisfy this M-of-N policy.”

So ownership is no longer just one person with one key.

16.2 Why hash the script?

Because the full script can be large.

So the blockchain output stores a compact hash-based commitment to the script, and the real script is revealed later when spending.

17) SegWit multisig format (P2WSH)

The format slide shows a SegWit multisig transaction format using **P2WSH** (Pay to Witness Script Hash). It has:

- version
- marker and flag
- inputs with empty ScriptSig
- outputs containing `0x00 0x20 <32-byte SHA256(witnessScript)>`
- witness containing:
 - count
 - `OP_0`
 - M signatures
 - witness script / redeem script:
`OP_M <pubkeys...> OP_N OP_CHECKMULTISIG`
- locktime.

This is the script-hash version of multisig.

17.1 What does P2WSH mean?

Pay to Witness Script Hash.

Instead of locking to a pubkey hash, the output locks to the **hash of a whole script**.

That script may encode a complex policy like 2-of-3 multisig.

17.2 Why use script hash?

Because the actual full multisig script may be too large to store directly in every output.

So the blockchain stores the compact hash, and later the spender reveals the real witness script when spending.

18) Multisig witness structure and OP_0 bug

The lecture's execution slides show the witness contains:

- `OP_0`
- `<sigA>`
- `<sigC>`
- `<witnessScript> OP_2 <PubKeyA> <PubKeyB> <PubKeyC> OP_3
OP_CHECKMULTISIG`

This `OP_0` is a famous Bitcoin quirk.

18.1 Why is `OP_0` there?

The lecture later says:

POP 0 (CHECKMULTISIG bug), then PUSH TRUE.

This means Bitcoin's `OP_CHECKMULTISIG` historically had an off-by-one bug and consumes an extra dummy item.

So an extra `0` is placed in witness/stack to satisfy that bug.

You do not need to derive the coding mistake — just remember:

multisig witness includes a dummy `OP_0` because of `CHECKMULTISIG` bug behavior.

That is very likely a short-answer detail.

19) Multisig execution — what happens conceptually?

The lecture's execution slides go step by step. Let's explain the whole story simply.

Step 1: Check witness script hash matches output

The lecture says compute `SHA256(witnessScript)` and check whether it equals the 32-byte value in the previous output's `scriptPubKey`. If not equal, fail. If equal, continue.

This proves the revealed witness script is exactly the one the old output committed to.

Step 2: Push witness items except last

The lecture says push all witness items except the last item, because the last item is the witness script itself.

So stack gets:

- `OP_0` dummy
- signatures

Step 3: Execute the witness script

Now the script runs:

`OP_2 <PubKeyA> <PubKeyB> <PubKeyC> OP_3 OP_CHECKMULTISIG`

This tells Bitcoin:

- $m = 2$
- $n = 3$
- here are the 3 pubkeys
- check whether 2 valid signatures are present

Step 4: Pop $n = 3$ and 3 pubkeys

The slides explicitly show:

- pop $n = 3$
- pop 3 public keys

Step 5: Pop $m = 2$ and 2 signatures

Then:

- pop $m = 2$
- pop 2 signatures

Step 6: Verify signatures against pubkeys

The slides show:

- $\text{verify}(\text{sigC}, [A,B,C]) \rightarrow \text{valid (matches C)}$
- $\text{verify}(\text{sigA}, [A,B,C]) \rightarrow \text{valid (matches A)}$
- $\text{count}(\text{valid}) = 2 = m \rightarrow \text{success}$

This means enough valid signatures matched the allowed key set.

Step 7: Pop dummy 0 and push TRUE

Because of the bug, CHECKMULTISIG pops the extra 0, then pushes TRUE if successful. The final slide explicitly says:

POP 0 (CHECKMULTISIG bug), then PUSH TRUE.

Step 8: Transaction validated

The lecture ends this sequence with “Transaction validated!!”

20) Multisig — the easiest possible understanding

A multisig output is like a digital locker that says:

“You need at least M approved keys out of N possible keys to open me.”

So in 2-of-3:

- key A + key B can spend
- key A + key C can spend
- key B + key C can spend
- but one key alone is not enough

This gives shared control and greater security.

Merkle trees and transaction verification

Now the lecture shifts to block-level transaction verification.

21) How do you verify a transaction inside a block?

The lecture gives a real scenario:

a vendor wants to verify a customer transaction before delivering merchandise. It gives a txid and block number, then lists the steps:

1. Check inclusion — confirm the transaction is part of the block
2. Validate block integrity — ensure the transaction contributes to the overall block hash
3. Confirm block validity — verify the block itself is valid and part of the longest chain.

This is very important.

21.1 Why is this needed?

Because it is not enough for a customer to just show you a txid string.

You need to know:

- the transaction is actually in a block
- the block is actually real and valid
- the block is actually on the accepted main chain

21.2 Three-layer idea

So verification means:

- **transaction inclusion**
- **block integrity**
- **chain validity**

That is a beautiful exam answer structure.

22) Block hash with naive transaction aggregation

The lecture shows a simple block hash picture:

- transactions Tx1, Tx2, Tx3, Txn
- each hashed
- combined into aggregate transaction hash
- along with prev block hash and nonce to make block hash.

Then it says if you verify some transaction inclusion directly this way, it has **$O(n)$** complexity.

22.1 Why $O(n)$?

Because if you did it naively, you might need to process or inspect all transactions to prove one specific transaction is included.

That becomes inefficient when blocks are large.

22.2 Why is this bad?

Because Bitcoin blocks can contain many transactions. Lightweight clients do not want to download everything just to verify one transaction.

So we need a better structure.

That leads to Merkle trees.

23) Merkle tree — the main idea

The lecture shows the Merkle tree:

- each transaction is hashed individually
- hashes are paired and hashed upward

- eventually one top value remains: the **Merkle root**
- the Merkle root goes into the block header.

This is one of the most beautiful data-structure ideas in blockchain.

23.1 What is the point?

Instead of combining all transactions in one flat expensive way, the transactions are organized in a binary hash tree.

Each parent hash summarizes two child hashes.

So the top root summarizes the whole transaction set compactly.

23.2 Why is this powerful?

Because you can prove one transaction's inclusion by showing only a small path of sibling hashes rather than the full block.

That is much more efficient.

24) Merkle proof and $O(\log n)$

The lecture then shows that verifying inclusion of, say, Tx2 in a block using the Merkle tree can be done in $O(\log n)$ instead of $O(n)$.

24.1 Why log n?

Because in a balanced binary tree, the path from one leaf to the root has logarithmic length.

So you only need the sibling hashes along that path.

24.2 Example idea

If you want to prove Tx2 is in the block, you do not need every transaction.

You need:

- Hash(Tx2)
- sibling hash at that level
- parent's sibling hash
- and so on up to the root

That sequence is called a **Merkle proof**.

24.3 Why does this matter?

Because lightweight clients can verify inclusion efficiently without downloading the whole block.

That is a huge scalability win.

25) Merkle tree benefits

The final slide summarizes Merkle trees as:

- summarizes transactions
- enables lightweight verification (SPV)
- detects tampering
- improves scalability
- proof size is small, about $\log_2(N)$ even for large blocks.

Let's make each one simple.

25.1 Summarizes transactions

All block transactions are compressed into one hash summary: the Merkle root.

25.2 Lightweight verification (SPV)

A light client can verify that a transaction is included using just a Merkle proof, not the whole block.

SPV = Simplified Payment Verification.

25.3 Detects tampering

If even one transaction changes, its hash changes, and therefore the Merkle root changes too.

That would invalidate the block header linkage.

25.4 Improves scalability

Instead of sending full block data for every small proof, the network can use tiny proofs.

So Merkle trees help blockchains scale better for verification.

Week 6 — Full connected story

Now let me connect the whole week into one story.

Week 6 begins by extending the legacy Bitcoin transaction model from Week 5. A single Bitcoin transaction may consume multiple earlier outputs and may create multiple new outputs, including payment outputs, change outputs, and even special unspendable outputs like OP_RETURN outputs. The lecture then zooms into the technical details of the signature inside scriptSig, explaining that Bitcoin legacy transactions use ECDSA signatures made of (r, s) and encoded in DER-like format with a final sighash byte. A crucial detail is that the same logical signature may admit different valid byte encodings, such as extra leading zeros.

That leads directly to **transaction malleability**. In legacy Bitcoin, txid is computed from the serialized transaction, and since scriptSig — including the signature bytes — is part of that serialization, a harmless change in signature encoding can change the txid without changing the actual economic effect of the transaction. This creates unreliable IDs, wallet confusion, network confusion, double-spend-style rebroadcast issues, and especially problems for higher-layer protocols that need stable txids. The lecture then shows the solution: Segregated Witness, which separates the witness (authorization proof) from the core transaction data used to compute txid. In SegWit, ScriptSig becomes empty and the signature/public key move into the witness section, making txids stable against this type of malleability.

The lecture then compares legacy P2PKH and SegWit P2WPKH. Even though SegWit outputs look different and have no obvious opcodes in the stored scriptPubKey, Bitcoin internally expands them into an equivalent virtual script that still asks the same core questions: does the provided public key hash match the expected hash, and is the signature valid? Witness items are pushed to the stack, opcodes are executed, and the final result becomes True if the spend is legitimate. So SegWit changes the transaction layout but not the fundamental ownership logic.

Lecture 12 then introduces **multisig**, where spending requires M valid signatures out of N possible keys. This is useful for shared accounts, escrow, stronger security, and Lightning-style channel constructions. In the P2WSH multisig version, the output stores only a hash of the full witness script. When spending, the witness reveals the script, signatures, and a dummy OP_0 needed because of the historical CHECKMULTISIG bug. Bitcoin first checks that the revealed witness script hashes to the expected script hash stored in the old output, then runs the script and verifies whether enough of the provided signatures match the authorized keys. If at least M valid signatures are found out of N possible keys, the script succeeds and the transaction is valid.

Finally, the lecture moves from transaction validation to block-level inclusion proofs. A vendor verifying a payment must not only know the txid, but must also confirm that the transaction is included in a valid block and that the block is part of the accepted longest chain. A naive inclusion proof over all transactions would be $O(n)$, so Bitcoin uses **Merkle trees**, where transaction hashes are combined upward into a Merkle root stored in the block header. This allows efficient $O(\log n)$ proofs of inclusion, supports SPV/lightweight clients, detects tampering, and improves scalability. So Week 6 completes the picture of how Bitcoin transactions are not only created and signed, but also encoded, protected against malleability, authorized under advanced policies like multisig, and efficiently proven to exist inside blocks.

Most important Week 6 topics to memorize

These are the highest-priority ones:

- multiple inputs and outputs
 - unspendable output / OP_RETURN
 - ECDSA signature format
 - r and s in signature
 - DER encoding idea
 - sighash byte
 - transaction malleability
 - why malleability is a problem
 - SegWit idea
 - legacy vs SegWit structure
 - P2WPKH
 - witness
 - virtual scriptPubKey
 - SegWit execution flow
 - multisig
 - M-of-N
 - P2WSH
 - witness script
 - CHECKMULTISIG bug / OP_0
 - verifying transaction inclusion in block
 - Merkle tree
 - Merkle root
 - SPV
 - $O(n)$ vs $O(\log n)$
-

Ultra-short Week 6 revision

A legacy Bitcoin transaction may have multiple inputs and outputs, and can even contain special unspendable outputs using OP_RETURN. Legacy signatures use ECDSA, where the signature consists of r and s encoded in DER format plus a sighash byte. Because legacy txid includes scriptSig, and scriptSig includes signature bytes, different valid encodings of the same signature can change the txid without changing the transaction's real effect — this is transaction malleability. SegWit fixes this by moving witness data like signatures and public keys outside the part used to compute txid. In P2WPKH, ScriptSig is empty, witness carries signature and public key, and Bitcoin internally expands the previous output into a virtual script equivalent to P2PKH for verification. Multisig transactions require M valid signatures out of N possible keys; in P2WSH the output stores a hash of the witness script, and the witness later reveals the script and signatures. Finally, Merkle trees summarize all block transactions into one Merkle root, enabling efficient $O(\log n)$ inclusion proofs, SPV, tamper detection, and better scalability.